



Facultad Politécnica, UNA
Departamento de Informática
Ingeniería Informática

Algoritmos y Estructura de Datos III

Unidad 2

ANÁLISIS DE ALGORITMOS

Objetivos

- Comprender el concepto de análisis de algoritmos y cálculo asintótico.
- Distinguir la aplicación de los distintos análisis de casos.
- Utilizar correctamente las notaciones asintóticas.
- Enunciar y comprender las propiedades de las notaciones asintóticas.
- Calcular analíticamente el tiempo de ejecución de algoritmos NO recursivos
- Estimar correctamente el requerimiento espacial de los algoritmos en términos asintóticos.

Contenido

- Motivación
- Introducción al análisis de Algoritmos
- Análisis de casos: mejor, peor y promedio
- Cálculo Asintótico
- Notaciones Asintóticas

¿Porqué no es suficiente encontrar un solo algoritmo para un problema determinado?

- RECURSOS LIMITADOS (procesador, memoria, ancho de banda, etc)
- TIEMPO DE RESPUESTA
- *¿Cómo escoger el algoritmo más conveniente?*
 - Buena codificación
 - Fácil de usar
 - Fácil de implementar
 - Eficiente (*¿qué significa eficiente?*)

Eficiencia de un algoritmo

Cuando se escribe la solución a un problema generalmente existen dos cosas inmersas en el diseño:

- Fácil de entender, codificar y depurar
(Ingeniería de Software).
- Uso eficiente de los recursos computacionales (tiempo y espacio)
(Análisis de algoritmos)

Cuando logro ambas cosas tengo un programa “elegante”.

Eficiencia de un algoritmo

¿Una máquina más rápida o
un algoritmo más eficiente?

Eficiencia de un algoritmo

Suponga el problema de ordenar un arreglo de números.

- Existen varios algoritmos, nosotros elegiremos dos para comparar:
 - Ordenación por inserción (***InsertionSort***)
 - Ordenación por mezcla (***MergeSort***).
- Ambos son algoritmos correctos.

Eficiencia de un algoritmo

Suponga que tiene dos máquinas **A** y **B**

- **A** puede ejecutar 1.000.000.000 (10^9) instrucciones por segundo
- **B** puede ejecutar 10.000.000 (10^7) instrucciones por segundo (*Note que es 100 veces más lenta que A*)
- c_1 es 2 y c_2 es 50 (constantes multiplicativas para **InsertSort** y **MergeSort** respectivamente). **¿Qué significa esto?**

Se quiere ordenar 1 millón de números ($n=1.000.000$)... la cuestión es *¿quién ganará en esta comparación?*

Eficiencia de un algoritmo

InsertSort toma alrededor de un tiempo de $c_1 n^2$ e en ordenar n items.

MergeSort toma alrededor de un tiempo de $c_2 n \log_2 n$ (log. en base 2)

c_1 y c_2 son factores constantes.

Consideramos que el tiempo es una función del tamaño de la entrada, normalmente lo denominados T .

Así el algoritmo de *InsertSort* toma un tiempo $T(n) = c_1 n^2$

InsertSort en la máquina A $c_1 n^2$

$$\frac{2 \cdot (10^6)^2 \text{ instrucciones}}{10^9 \text{ instrucciones/segundo}} = 2000 \text{ segundos}$$

33 minutos, 20 segundos

MergeSort en la máquina B $c_2 n \log_2 n$

$$\frac{50 \cdot (10^6) \log 10^6 \text{ instrucciones}}{10^7 \text{ instrucciones/segundo}} \approx 100 \text{ segundos}$$

1 minuto, 40 segundos

- ¿Qué pasa cuando n = diez millones? 2d, 7h, 33m, 20s vs 19m, 22s
- ¿Con cuál algoritmo se quedaría?
- ¿Cuánto tardará *InsertSort* si ahora la máquina A es 10 veces más rápida?
- ¿Cuál es el efecto de la rapidez de la máquina?

Efecto del algoritmo y la rapidez de la máquina

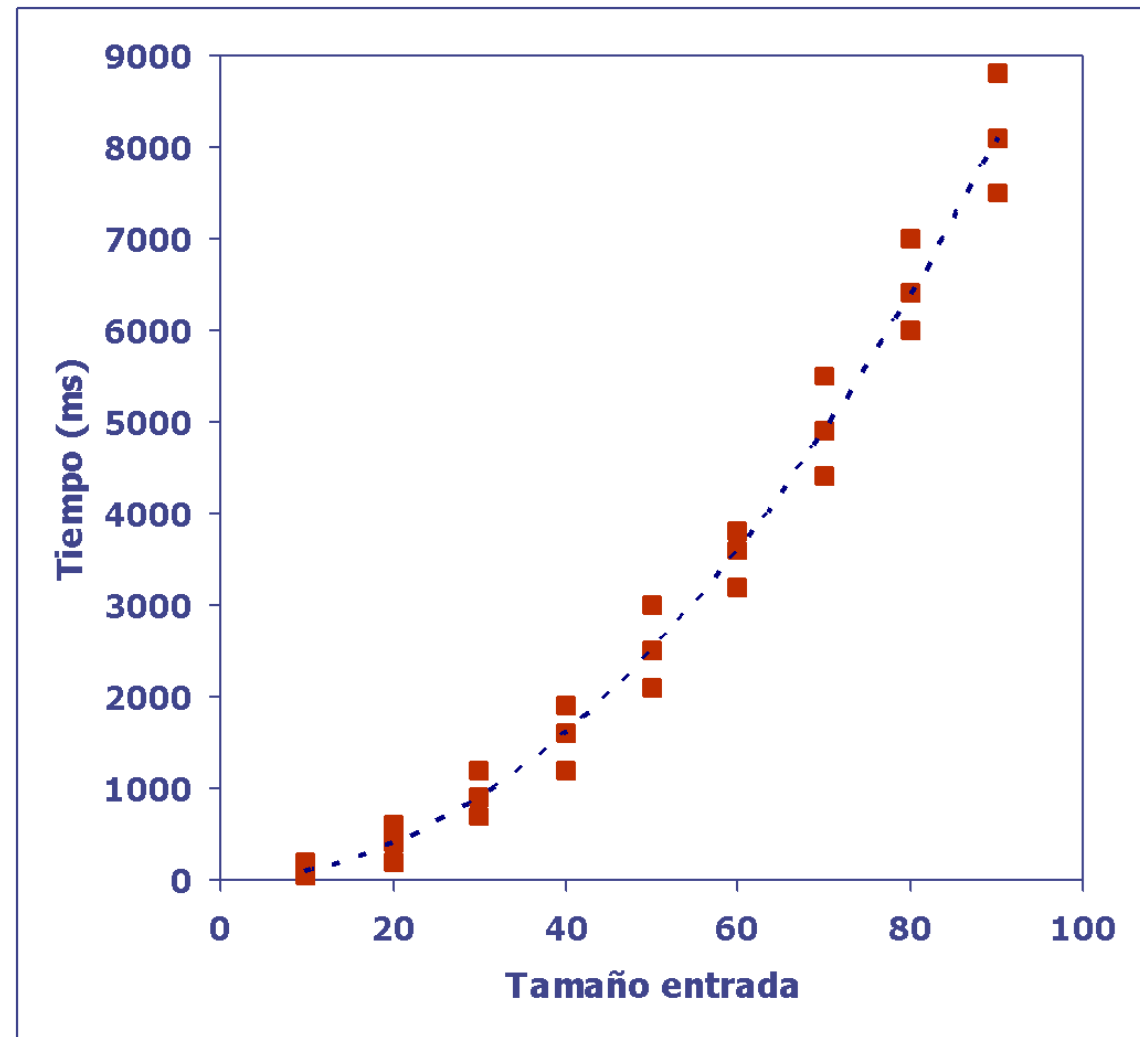
Tiempo de ejecución del algoritmo - $T(n)$	Máximo tamaño solucionable en 1 segundo		
	Computador actual	100 veces más rápida	1000 veces más rápida
n	$N_0=100$ millones	$100N_0$	$1000N_0$
$100n$	$N_1=1$ millón	$100N_1$	$1000N_1$
n^2	$N_2=10000$	$10N_2$	$31.6N_2$
n^3	$N_3=464$	$4.64N_3$	$10N_3$
2^n	$N_4=26$	$N_4 + 6.64$	$N_4 + 9.97$

¿Cómo escoger el mejor algoritmo?

- Medir el tiempo de respuesta.. pero dependo de:
 - Del subconjunto de pruebas escogidas
 - Del ordenador donde hago las pruebas
 - Del lenguaje de programación utilizado
 - Del compilador utilizado
 - De la calidad de la programación
- Para hacer esto tengo que codificar/programar y en ocasiones esto no es trivial, no es sencillo.

¿Cómo escoger el mejor algoritmo?

- *Escribo el programa*
- *Ejecuto el programa*
- *Mido el tiempo*
- *Grafico el resultado*
- *Encuentro la tendencia*



¿Cómo escoger el mejor algoritmo?

Otra alternativa:

- Predecir el comportamiento del algoritmo sin necesidad de implementarlo.
- Para ello nos ayudará el ***análisis de algoritmo*** y el ***cálculo asintótico***.

Análisis de Algoritmos y Cálculo asintótico

- El **análisis de algoritmos** estudia desde el punto de vista teórico, los recursos computacionales que necesita la ejecución de un programa de computadora: *su eficiencia*.
- *Aparte del tiempo de cómputo, ¿Cuáles son otros recursos?*
- Es una técnica de estimación. Es particularmente útil para determinar si vale o no la pena implementar un algoritmo en particular.
- El **cálculo asintótico** es una técnica de caracterización de los algoritmos para poder luego compararlos.

Análisis de Algoritmos

- Utilizamos una descripción de alto nivel para el algoritmo (*generalmente*)
- Caracterizamos el tiempo de ejecución como una función al tamaño de la entrada N , $T(N)$
- Se toma en cuenta todas las posibles entradas (*instancias del problema*)
- Permite evaluar la velocidad del algoritmo independientemente del entorno de HW o SW.

Análisis de Algoritmos

El modelo RAM (*Random Access Machine*)

- Las instrucciones son ejecutadas una tras otra sin concurrencia. **Es un modelo secuencial.**
- Un banco de celdas de memoria potencialmente ilimitado.
- Las celdas de memoria están numeradas y accederlas toma una unidad de tiempo constante.
- Las operaciones son similares a cualquier computadora tradicional.
- Cada operación simple o elemental (**OE**) (+, *, -, =, jump, call, ..) toma exactamente 1 unidad de tiempo.

Importante: esto es una simplificación de la realidad

¿Por qué le parece? ¿Se imagina otro modelo diferente?

- ¿Cómo analizamos el tiempo de ejecución?
- El tiempo depende de la entrada
- El tamaño de la entrada depende del problema estudiado (ordenación, multiplicar dos enteros, algoritmos en grafos, etc)
- ***Tiempo de ejecución:*** para una entrada particular ***es el número de OE ejecutadas*** (o número de instrucciones).

Análisis de Algoritmos

Tipos de análisis

- **Peor caso** (usual) (*worst*)

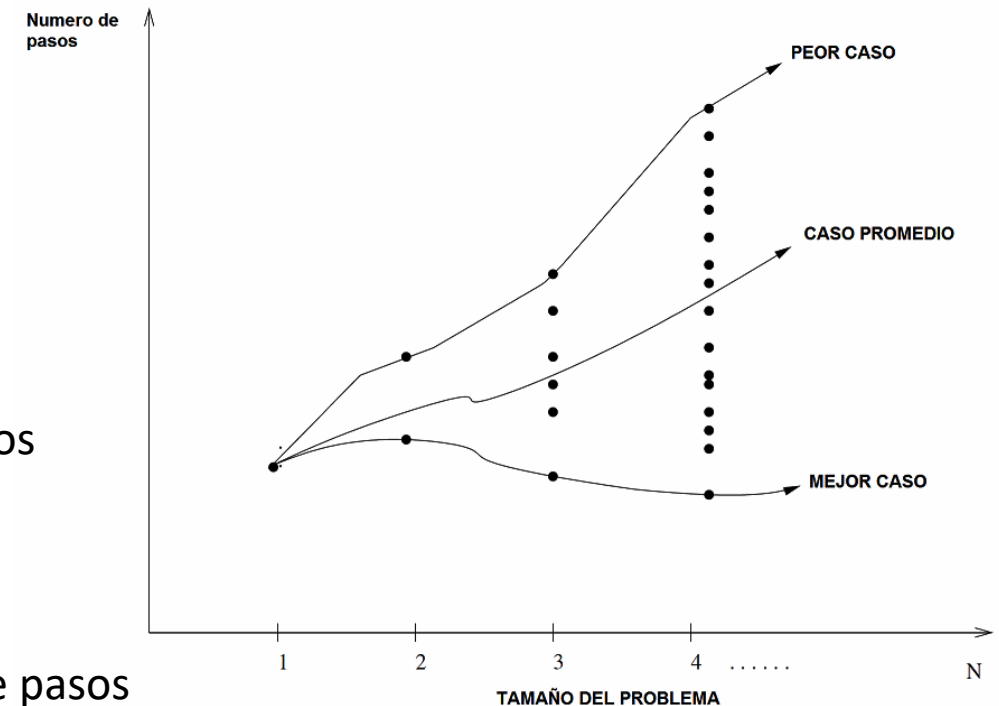
- Es la función definida por el máximo número de pasos que toma cualquier instancia de tamaño N

- **Caso promedio** (*average*)

- Es la función definida por el promedio de número de pasos que toma cualquier instancia de tamaño N . Implica estudio estadístico de la distribución de las instancias del problema analizado.
- Es el mejor estudio aunque su cálculo es más complejo.

- **Mejor caso** (*best*)

- Es la función definida por el mínimo número de pasos que toma cualquier instancia de tamaño N . Es el caso más engañoso



Ejemplo 1 de análisis de casos

Considere el algoritmo de buscar un elemento en un arreglo desordenado:

- ¿Cuál es el peor caso?
- ¿Cuál es el mejor caso?
- ¿Cuál es el caso promedio?
- ¿Cómo calcularía el tiempo para cada caso?

[16, 20, 30, 50, 26, 17, 59, 89, 78, 65, 7, 8, 1, 4, 5]

↑
Buscamos el 16

↑
Buscamos el 5

Ejemplo 2 de análisis de casos

Considere ahora el algoritmo de ordenación por inserción. ¿Cuál es el mejor, peor y caso promedio?

```
INSERT-SORT(A)  // Arreglo A[1..n]
```

```
for  $j \leftarrow 2$  to  $n$  {
```

```
     $key \leftarrow A[j]$ 
```

```
    // Inserto  $A[j]$  en la secuencia ordenada  $A[1..j-1]$ 
```

```
     $i \leftarrow j-1$ 
```

```
    while  $i > 0$  and  $A[i] > key$  {
```

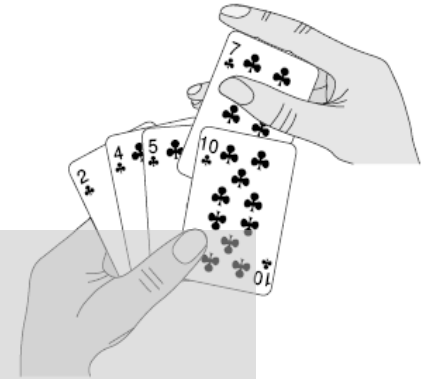
```
         $A[i+1] \leftarrow A[i]$ 
```

```
         $i \leftarrow i - 1$ 
```

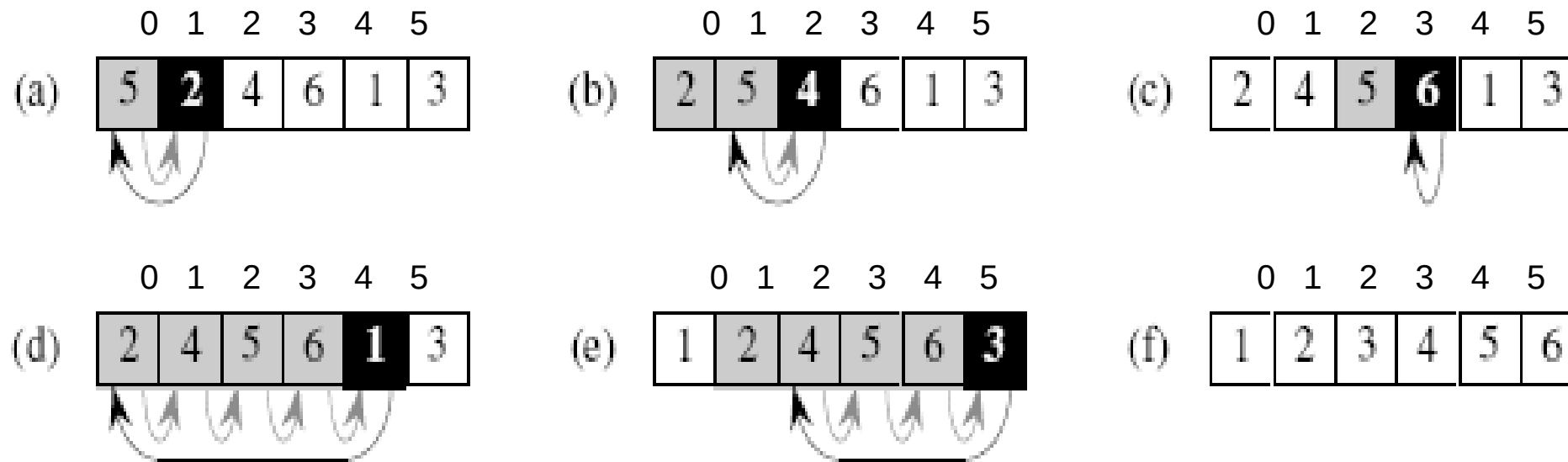
```
    }
```

```
     $A[i+1] \leftarrow key$ 
```

```
}
```



Funcionamiento de InsertSort



$A = \{ 5, 2, 4, 6, 1, 3 \}$

Ejemplo 2 de análisis de casos

- Mejor caso del algoritmo
 - Cuando no se ingresa al ciclo mientras y ocurre **¿cuándo?**
- Caso promedio
 - Cuando se ingresa la mitad de las veces... aunque depende..**¿de qué?**
- Peor caso
 - Cuando siempre se ingresa.

ESTIMACIÓN DE TIEMPO

Cálculo de tiempo de ejecución

Consideraciones sobre las operaciones

- Una asignación = 1 OE (Operación elemental)
- Una comparación = 1 OE
- Acceso a arreglo = 1 OE
- Llamada a función = 1 OE
- Una operación simple = 1 OE (resta, suma, .., etc)
- Consideramos todas las operaciones del mismo costo (*aunque no es real, ¿por qué?*)

Cálculo de tiempo de ejecución

Inspeccionando el código podemos determinar el número de operaciones elementales. Analicemos el siguiente algoritmo, encontrar el máximo elemento de en un arreglo (*Recibimos el vector A y su tamaño n*)

Algoritmo **findMax**(A, n)// Arreglo $A[1..n]$

Operaciones

$Maximo \leftarrow A[1]$	2
for $i \leftarrow 2$ to n do	$1 + n$
if $A[i] > Maximo$ then	$2(n-1)$
$Maximo \leftarrow A[i]$	$2(n-1)$ (peor caso)
{ incremento del contado i }	$2(n-1)$
return $Maximo$	1
Total	$7n - 2$

Cálculo de tiempo de ejecución

- El algoritmo ***findMax*** ejecuta $7n-2$ operaciones en el peor caso. Definimos entonces :
 - a = Tiempo que toma la operación elemental más rápida
 - b = Tiempo que toma la operación elemental más lenta
- Decimos que $T(n)$ es el tiempo para ***findMax***. Entonces

$$a(7n - 2) \leq T(n) \leq b(7n - 2)$$

De aquí concluimos que el tiempo de ejecución $T(n)$ está limitado por dos funciones lineales.

Cálculo de tiempo de ejecución

- El cambio del entorno de HW o SW
 - Afecta a $T(n)$ en un factor constante, pero
 - NO afecta la tasa de crecimiento de $T(n)$
- Por tanto, la tasa de crecimiento lineal del tiempo de ejecución de $T(n)$ es una propiedad intrínseca del algoritmo ***findMax***

Ejemplo de análisis de InsertSort

Algoritmo *InsertSort*(**A**, **n**) // Arreglo **A**[1..**n**]

Operaciones

for $j \leftarrow 2$ to $length(A)$ do	----->	$1 + n$
$key \leftarrow A[j]$	----->	$2 (n - 1)$
$i \leftarrow j - 1$	----->	$2 (n - 1)$
		n
while $i > 0$ and $A[i] > key$ do	----->	$4. \sum_{j=2}^n t_j$
		n
$A[i + 1] \leftarrow A[i]$	----->	$6. \sum_{j=2}^n (t_j - 1)$
$i \leftarrow i - 1$	----->	
 $key \leftarrow A[j]$	----->	$2 (n - 1)$
{ incremento del contador j }	----->	$2 (n - 1)$

$$\text{TOTAL} \longrightarrow 4. \sum_{j=2}^n t_j + 6. \sum_{j=2}^n (t_j - 1) + 9n - 5$$

Ejemplo de análisis de InsertSort

- **Mejor caso:**

- Arreglo ordenado . $t_j = 1$ para $j=2,3,..,n$

$$T(n) = 4 \cdot (n - 1) + 9n - 5$$

$$T(n) = 13n - 9$$

- **Peor caso:**

- Arreglo ordenado en forma reversa. $t_j = j$ para $j=2,3,..,n$

$$T(n) = 4 \cdot \left(\frac{n(n+1)}{2} - 1 \right) + 6 \cdot \left(\frac{n(n-1)}{2} \right) + 9n - 5$$

$$T(n) = 5n^2 + 7n - 9$$

Cálculo asintótico

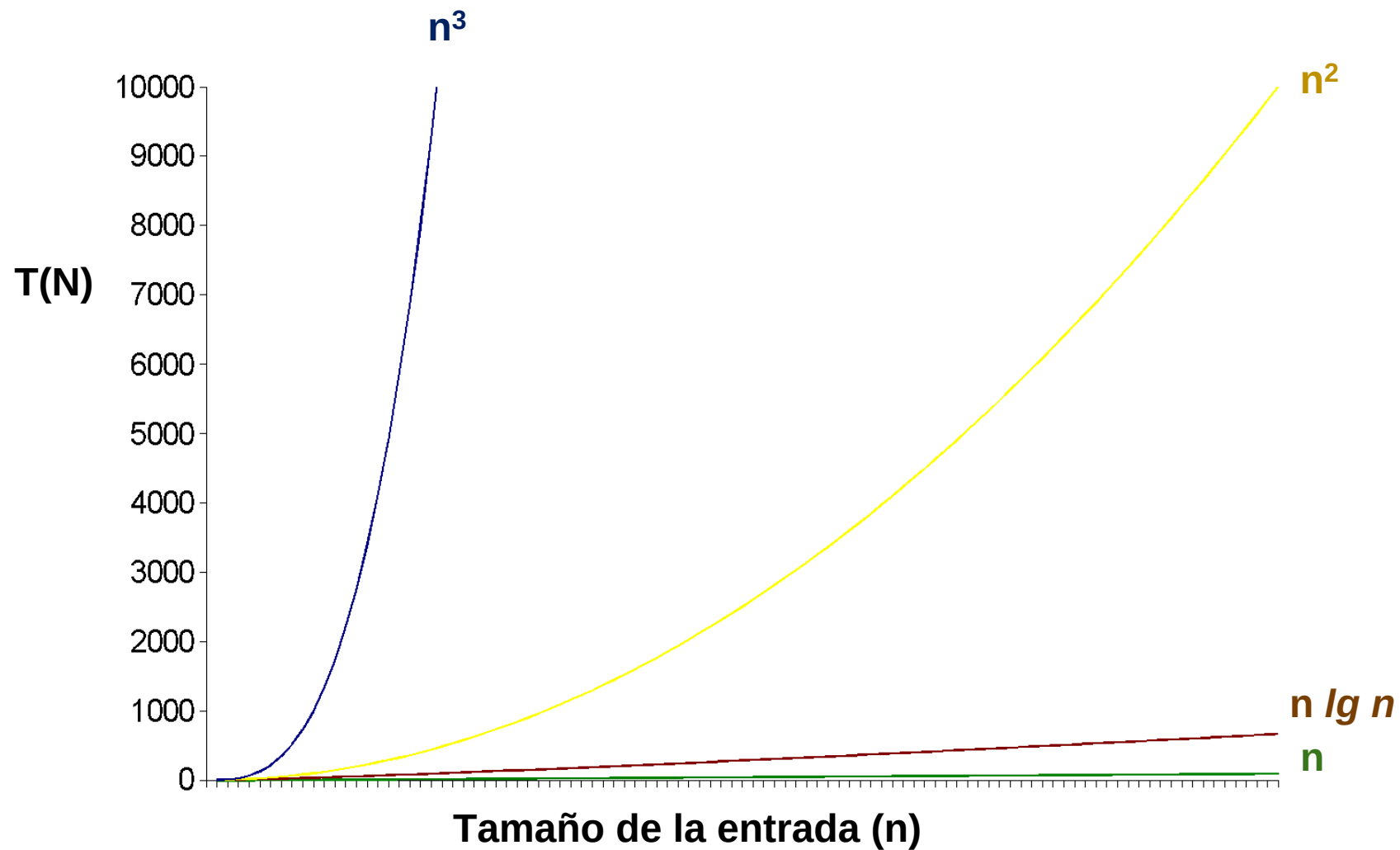
- El cálculo asintótico¹ de un algoritmo me permite clasificarlo de acuerdo a su “tasa de crecimiento” (*growth rate*).
- Ignoramos las constantes que puedan tenerse y nos concentramos en el grado dominante. De esta forma puedo comparar algoritmos de una forma más sencilla.

¹ Propuesto por **Donal Knuth** en su libro “El arte de programar computadores” (primera edición : 1968)

Tasas de crecimiento usuales

<u>Clase</u>	<u>Nombre de la función</u>	
c	Constante	 Crecimiento
$\lg n$	Logarítmica	
$\lg^2 n$	Logarítmica al cuadrado	
n	Lineal	
$n \lg n$	" $n \lg n$ " o <i>polilogarítmica</i>	
n^2	Cuadrática	
n^3	Cúbica	
$n^m, m=0,1,2,3,..$	Polinomial	
$c^n, c>1$	Exponencial	
$n!$	Factorial	

Crecimiento de algunas funciones



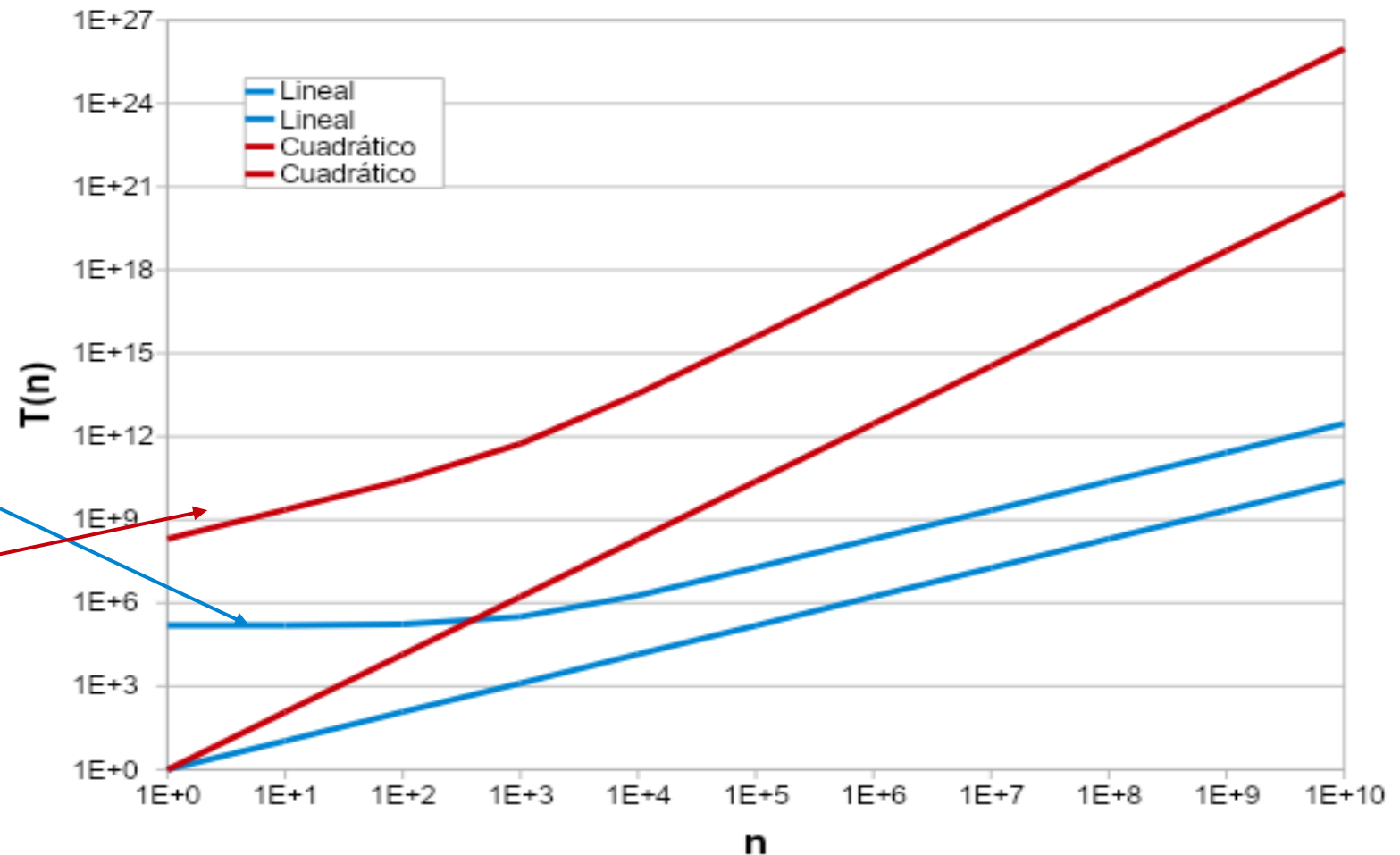
Tasas de crecimiento

- No es afectada por factores constantes o por términos de menor grado.

Ejemplos:

- $10^2 n + 10^5$ tiene una tasa de crecimiento lineal

- $10^5 n^2 + 10^8 n$ tiene una tasa de crecimiento cuadrática.



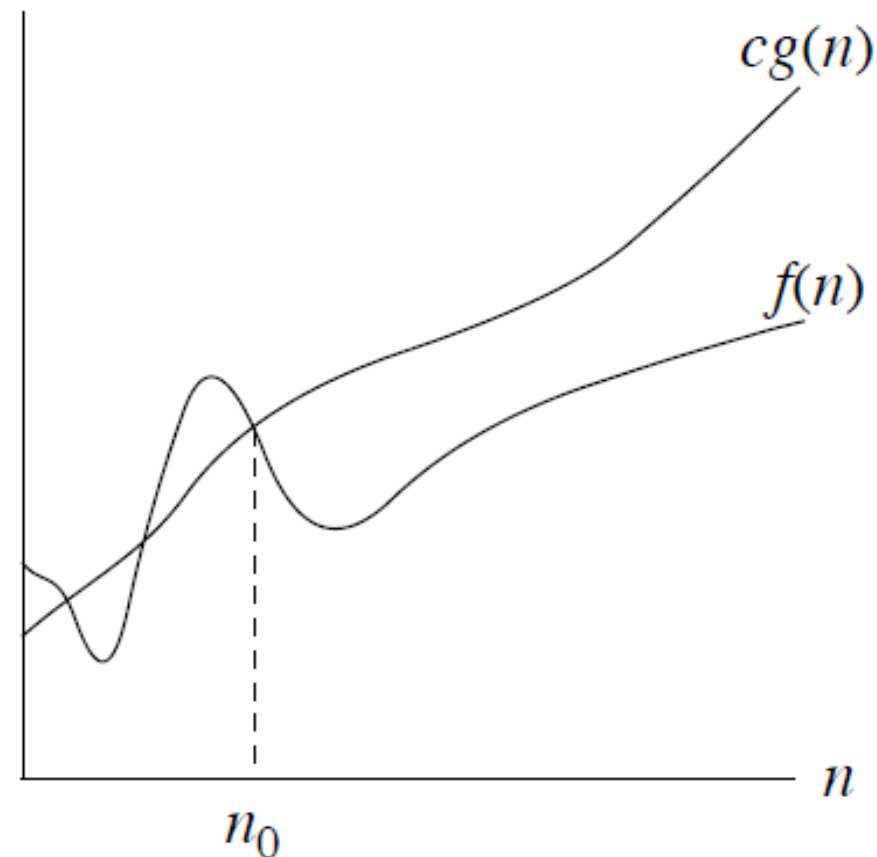
Análisis asintótico

- Para realizar el análisis debemos:
 - Encontrar el número de operaciones en el peor caso
 - Expresar esta función en una notación asintótica, por el momento en la notación O (*o grande*)
- Ejemplo:
 - Nosotros determinamos que ***findMax*** ejecuta en a lo sumo $7n-2$ operaciones primitivas.
 - Entonces decimos que el algoritmo ***findMax*** “está en $O(n)$ ”

Notación O (o grande) (Cota superior)

Dada las funciones $f(n)$ y $g(n)$, se dice que $f(n)$ “está en $O(g(n))$ ” si existen las constantes positivas c y n_0 tal que

$$f(n) \leq cg(n) \\ \text{para } n \geq n_0$$



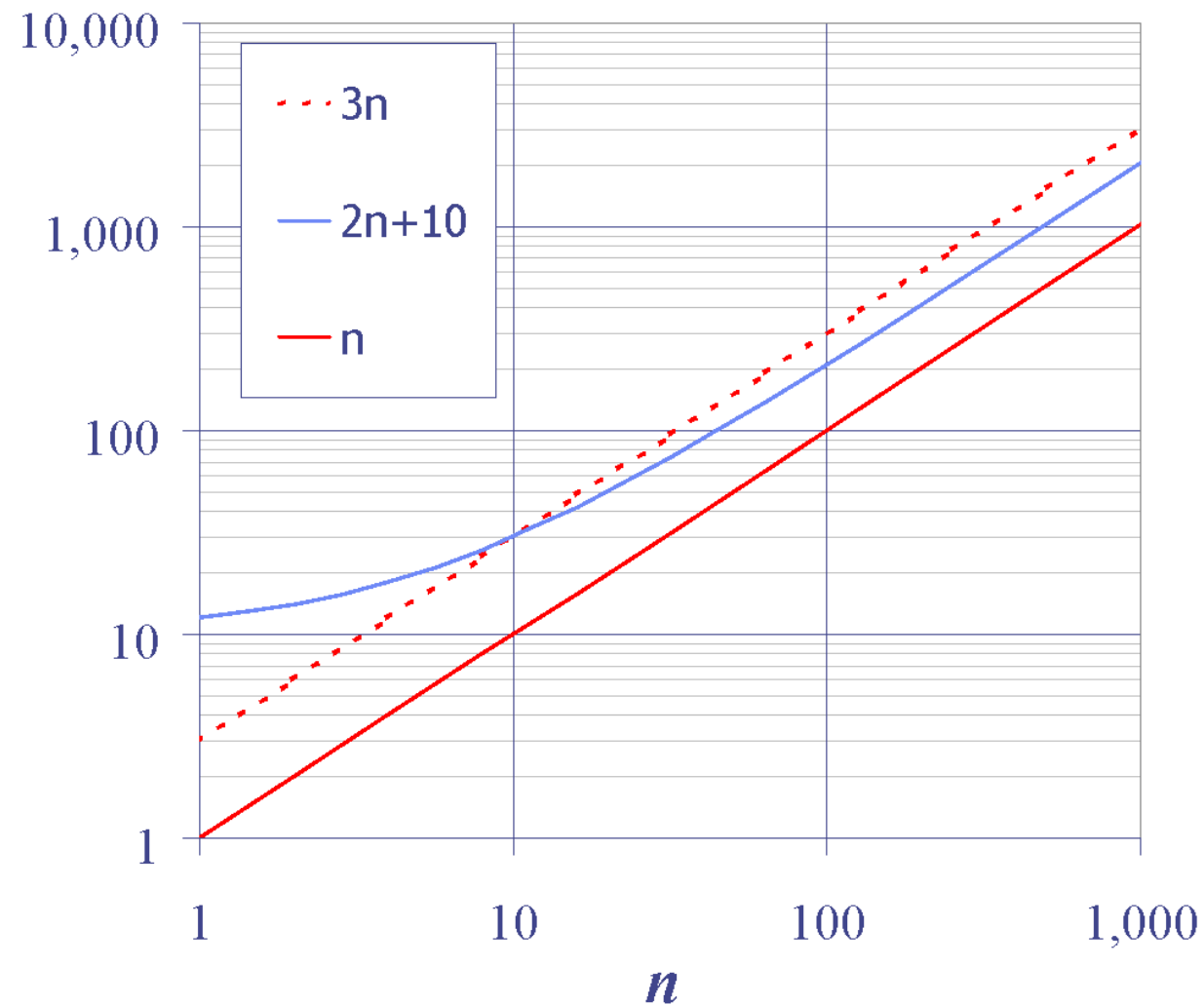
Notación O

Ejemplo 1:

$2n+10$ *esta en* $O(n)$

$$\begin{aligned} 2n+10 &\leq cn \\ (c-2)n &\geq 10 \\ n &\geq 10/(c-2) \end{aligned}$$

$$c=3 \text{ y } n_0=10$$



Notación O

Ejemplo 2 :

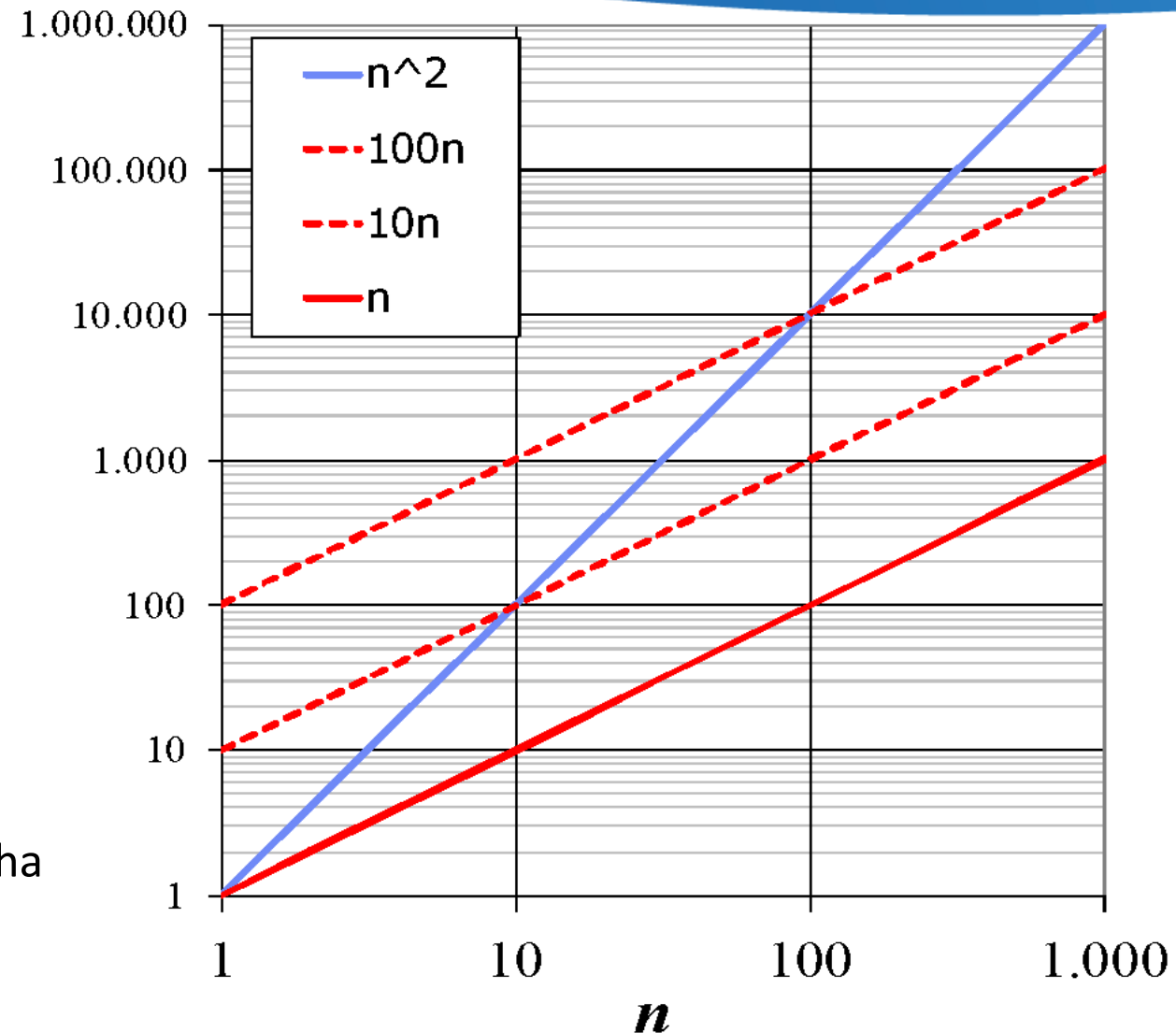
La función

n^2 no está en $O(n)$

$$n^2 \leq cn$$

$$n \leq c$$

La desigualdad no puede ser satisfecha
ya que C debe ser una constante



Ejemplos de notación O

- $7n-2$ está en $O(n)$
- $3n^3 + 20n^2 + 5$ está en $O(?)$
- $3 \lg n + \lg \lg n$ está en $O(?)$

Reglas de la notación O

- Si es $f(n)$ un polinomio de grado d , entonces $f(n)$ están en $O(n^d)$, esto implica:
 - Extraer los términos de menor orden
 - Extraer los factores constantes
- Utilizar la clase de función más ajustada
 - Decir $2n$ está en $O(n)$ y no $2n$ está en $O(n^2)$
- Utilizar la expresión simple de la clase
 - Decir $3n + 5$ está en $O(n)$ y **NO** $3n+5$ está en $O(3n)$

Otras notaciones asintóticas

Notación Ω (Omega) (*cota inferior*)

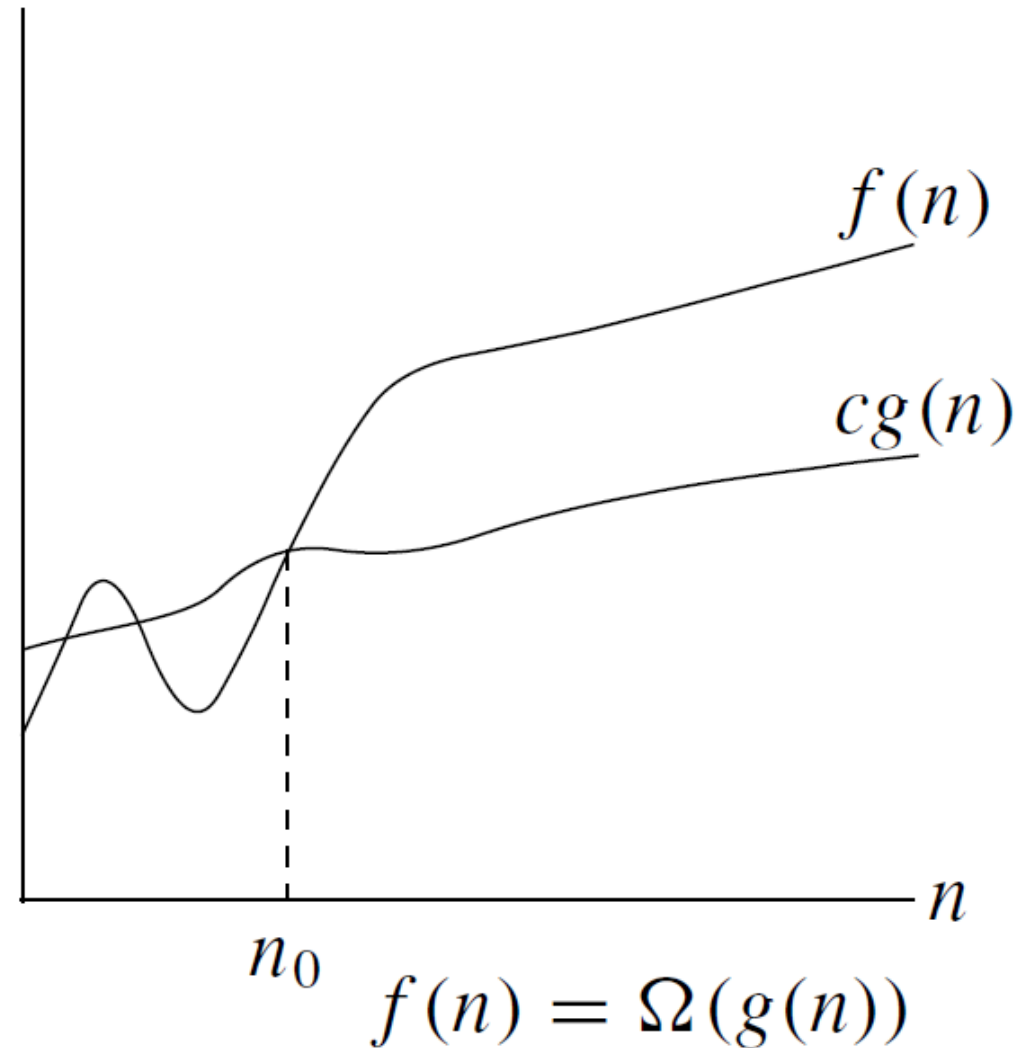
$f(n)$ está en $\Omega(g(n))$ si existe una constante c y un entero constante $n_0 \geq 1$ tal que

$$f(n) \geq c \cdot g(n) \text{ para } n \geq n_0$$

Ejemplos:

$$\sqrt{n} = \Omega(\log n)$$

$$10n^2 = \Omega(n^{3/2})$$



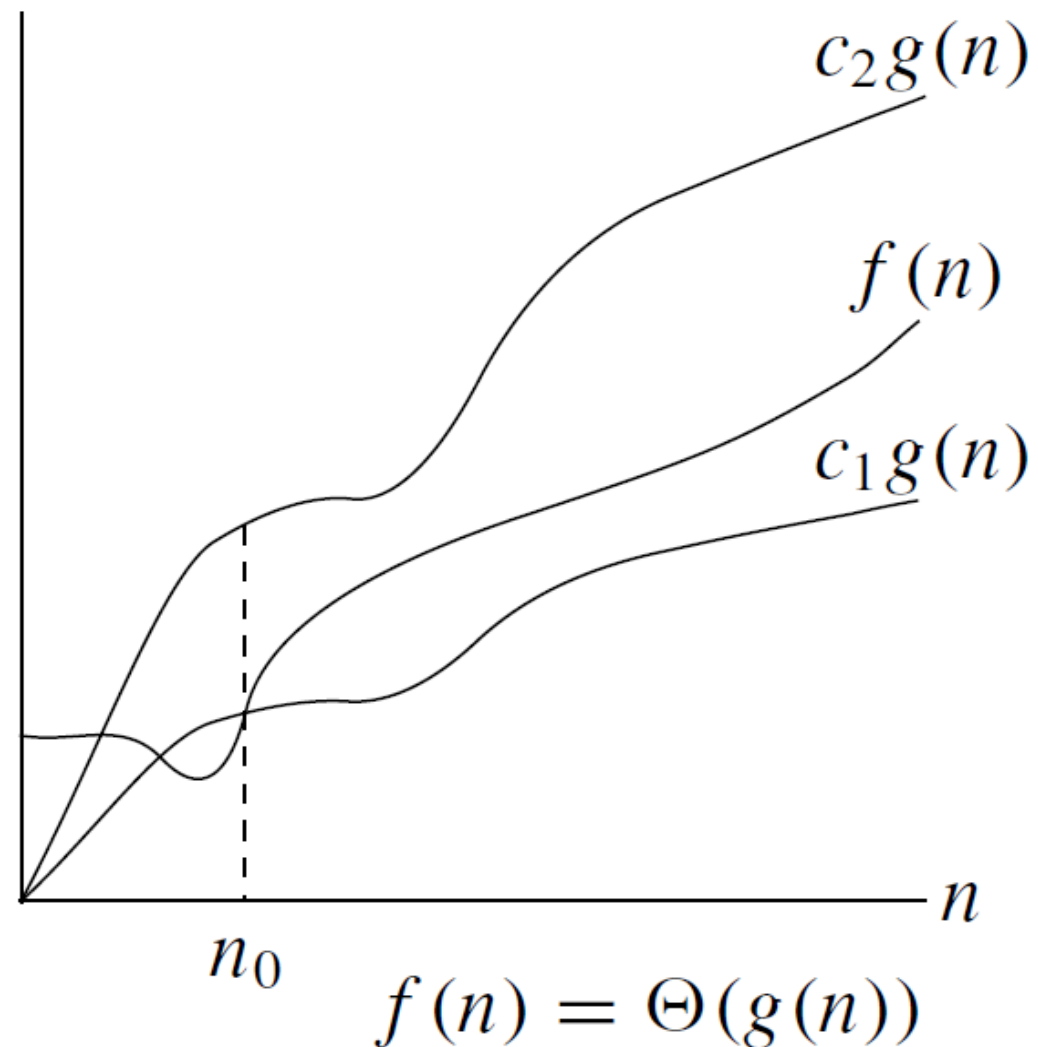
Otras notaciones asintóticas

Notación Θ (theta)

$f(n)$ está en $\Theta(g(n))$ si existen las constantes c_1 y c_2 y un entero constante $n_0 \geq 1$ tal que

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

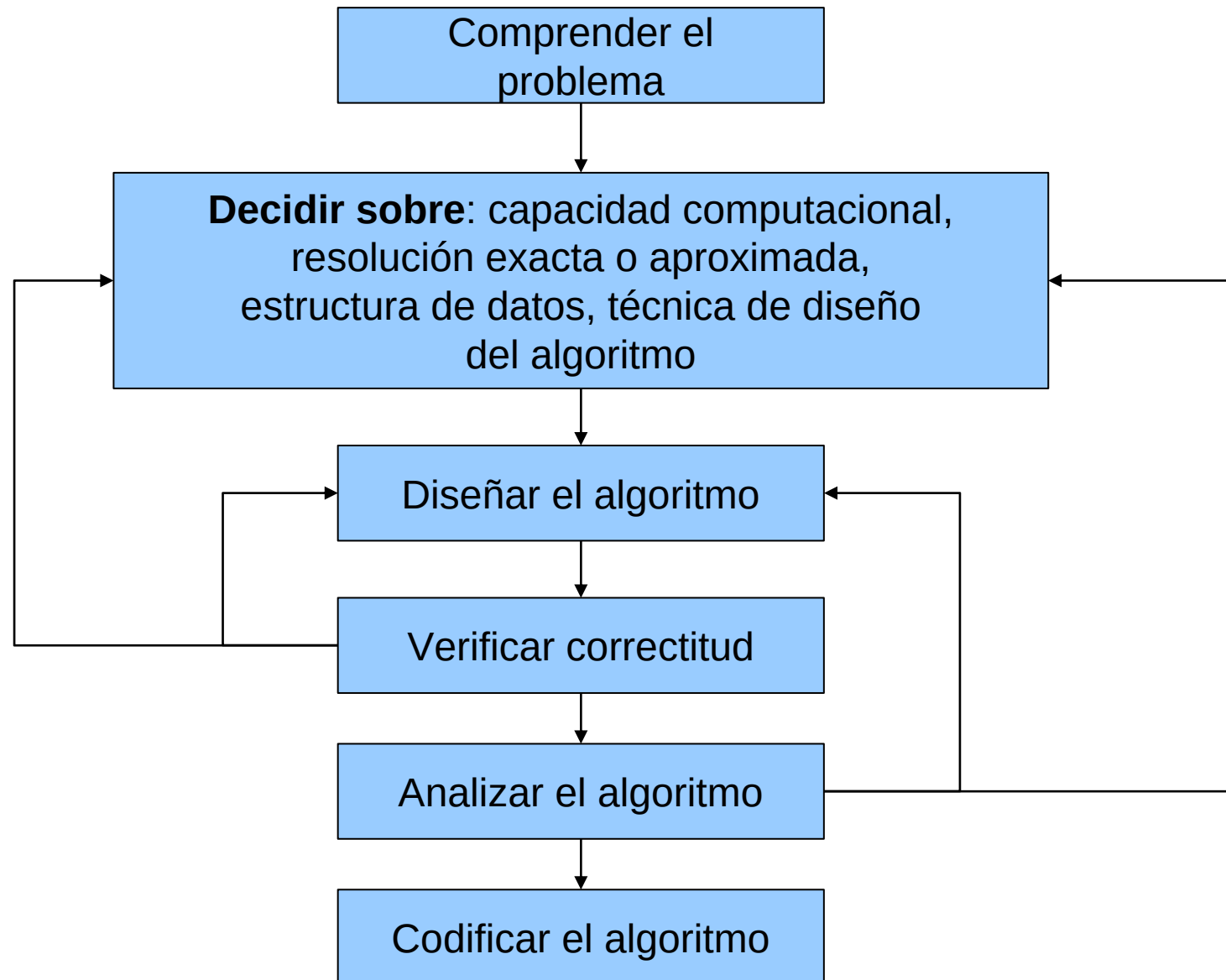
para $n \geq n_0$



Notaciones asintóticas, visión intuitiva

- O grande
 - $f(n)$ está en $O(g(n))$ si $f(n)$ es asintóticamente **menor o igual** que $g(n)$
- Ω grande
 - $f(n)$ está en $\Omega(g(n))$ si $f(n)$ es asintóticamente **mayor o igual** a $g(n)$
- Θ grande
 - $f(n)$ está en $\Theta(g(n))$ si $f(n)$ es asintóticamente **igual** a $g(n)$

Diseño de un algoritmo y proceso de análisis



Otras notaciones asintóticas

Notación o (o chica)

Dada las funciones $f(n)$ y $g(n)$, se dice que $f(n)$ “está en $o(g(n))$ ” si existen las constantes positivas $c > 0$ y n_0 tal que

$$f(n) < cg(n) \text{ para } n \geq n_0$$

Por ejemplo $2n = o(n^2)$ pero $2n^2 \neq o(n^2)$

Otros ejemplos:

$$n^{1.9999} = o(n^2)$$

$$n^2 / \lg n = o(n^2)$$

$$n^2 / 1000 \neq o(n^2)$$

Otras notaciones asintóticas

Notación ω (omega chica)

Dada las funciones $f(n)$ y $g(n)$, se dice que $f(n)$ “está en $\omega(g(n))$ ” si existen las constantes positivas $c > 0$ y n_0 tal que

$$f(n) > cg(n) \text{ para } n \geq n_0$$

Por ejemplo $n^2/2 = \omega(n)$ pero $n^2/2 \neq \omega(n^2)$

Otros ejemplos:

$$n^{2.0001} = \omega(n^2)$$

$$n^2 / \lg n = \omega(n^2)$$

$$n^2 \neq \omega(n^2)$$

Límites para comparar tasas de crecimiento

Aunque las definiciones de $O, \Theta, \Omega, o, \omega$ son indispensables, un método simple para comparar las tasas de crecimiento de dos funciones es el uso del cálculo del límite del ratio de estas funciones. Los tres casos son:

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \begin{cases} 0 & t(n) < g(n) \\ c > 0 & t(n) = g(n) \\ \infty & t(n) > g(n) \end{cases}$$

Note que los dos primeros casos significan que $t(n) \in O(g(n))$

los dos últimos casos significan que $t(n) \in \Omega(g(n))$

y el segundo caso significa que $t(n) \in \Theta(g(n))$

Aplicar Límites

- Puede utilizar la regla de *L'Hôpital-Bernoulli*

$$\lim_{n \rightarrow \infty} \frac{t(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{t'(n)}{g'(n)}$$

- Y la fórmula de Stirling para $n!$

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e} \right)^n$$

Comparar orden de crecimiento (Ejemplos)

$$a) \quad \frac{1}{2}n(n-1) \quad \text{vs} \quad n^2$$

$$b) \quad \log_2 n \quad \text{vs} \quad \sqrt{n}$$

$$c) \quad n! \quad \text{vs} \quad 2^n$$

$$a) \quad \lim_{n \rightarrow \infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n \rightarrow \infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2} \Rightarrow \frac{1}{2}n(n-1) \in \Theta(n^2)$$

$$b) \quad \lim_{n \rightarrow \infty} \frac{\log_2 n}{\sqrt{n}} = \lim_{n \rightarrow \infty} \frac{(\log_2 n)'}{(\sqrt{n})'} = \lim_{n \rightarrow \infty} \frac{(\log_2 e) \frac{1}{n}}{\frac{1}{2\sqrt{n}}} = 2 \log_2 \lim_{n \rightarrow \infty} \frac{\frac{1}{n}}{\frac{1}{\sqrt{n}}} = 2 \log_2 \lim_{n \rightarrow \infty} \frac{(\frac{1}{\sqrt{n}})'}{(\frac{1}{n})'}$$

$$= 2 \log_2 e \lim_{n \rightarrow \infty} \frac{1}{2\sqrt{n}} = 0 \Rightarrow \log_2 n \in O(\sqrt{n}) \Rightarrow \log_2 n \in o(\sqrt{n})$$

$$c) \quad \lim_{n \rightarrow \infty} \frac{n!}{2^n} = \lim_{n \rightarrow \infty} \frac{\sqrt{2\pi n} \left(\frac{n}{e}\right)^n}{2^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \frac{n^n}{2^n e^n} = \lim_{n \rightarrow \infty} \sqrt{2\pi n} \left(\frac{n}{2e}\right)^n = \infty \Rightarrow n! \in \Omega(2^n) \Rightarrow n! \in \omega(2^n)$$

Definiciones alternativas de cotas

$O(f(n))$ sii

$$\lim_{n \rightarrow \infty} \frac{t(n)}{f(n)} = C, \quad 0 \leq C < \infty$$

$\Theta(f(n))$ sii

$$\lim_{n \rightarrow \infty} \frac{t(n)}{f(n)} = C, \quad 0 < C < \infty$$

$\Omega(f(n))$ sii

$$\lim_{n \rightarrow \infty} \frac{t(n)}{f(n)} = C, \quad 0 < C \leq \infty$$

$o(f(n))$ sii

$$\lim_{n \rightarrow \infty} \frac{t(n)}{f(n)} = 0$$

$\omega(f(n))$ sii

$$\lim_{n \rightarrow \infty} \frac{t(n)}{f(n)} = \infty$$

Algunas propiedades de las cotas

- $f(n) \in O(f(n))$ (reflexividad – aplicable a las otras cotas)
- $f(n) \in \Theta(g(n))$ **sii** $g(n) \in \Theta(f(n))$ (simetría)
- $f(n) \in O(g(n))$ **sii** $g(n) \in \Omega(f(n))$ (simetría traspuesta)
- **Si** $f(n) \in O(g(n))$ y $g(n) \in O(h(n))$, **entonces**
 $f(n) \in O(h(n))$ (transitividad – aplicable a las otras cotas)
- **Si** $f_1(n) \in O(g_1(n))$ y $f_2(n) \in O(g_2(n))$, **entonces**
 $f_1(n) + f_2(n) \in O(\mathbf{max}\{g_1(n), g_2(n)\})$
- **Si** $f(n) \in O(g(n))$ y $f(n) \in \Omega(g(n))$, **entonces** $f(n) \in \Theta(g(n))$

Ejemplo de notaciones asintóticas

Ejemplos de funciones en $O(n^2)$	Ejemplo de funciones en $\Omega(n^2)$
n^2	n^2
$n^2 + n$	$n^2 + n$
$n^2 + 1000n$	$n^2 - n$
$1000 n^2 + 1000 n$	$1000 n^2 + 1000 n$
También	$1000 n^2 - 1000 n$
n	También
$n/1000$	n^3
$n^{1.9999}$	$n^{2.0001}$
$n^2 / \lg \lg \lg n$	$n^2 \lg \lg \lg n$
	2^{2n}

O, Θ, Ω como funciones anónimas

- Podemos usar las notaciones asintóticas para representar funciones desconocidas. Ejemplo:
 - $f(n) = 10n^2 + O(n)$
- Significa que $f(n)$ es igual a $10n^2$ más una función que no conocemos o que no nos importa conocer y que es asintóticamente al menos lineal en n .
- Ejemplos:
 - $n^2 + 4n - 1 = n^2 + \theta(n)?$ ✓
 - $n^2 + \Omega(n) - 1 = O(n^2)?$ ✗
 - $n^2 + O(n) - 1 = O(n^2)?$ ✓
 - $n \log n + \theta(\sqrt{n}) - 1 = O(n\sqrt{n})?$ ✓

Recordar que

$O \approx \geq$
$\Omega \approx \leq$
$\Theta \approx =$
$O \approx >$
$\omega \approx <$

Series útiles para el análisis

$$\sum_{i=1}^n 1 = n \in \Theta(n)$$

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \in \Theta(n^2)$$

$$\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6} \in \Theta(n^3)$$

$$\sum_{i=1}^n a^i = \frac{a^{n+1} - 1}{a - 1} \quad \text{para } a > 1$$

$$\text{En particular } \sum_{i=1}^n 2^i = 2^{n+1} - 1 \in \Theta(2^n)$$

Serie Armónica

$$\sum_{i=1}^n \frac{1}{i} = \ln n + O(1)$$

Algunas propiedades básicas de las sumatorias

$$\sum (a_i \pm b_i) = \sum a_i \pm \sum b_i \quad \left| \quad \sum c a_i = c \sum a_i \quad \left| \quad \sum_{l \leq i \leq u} a_i = \sum_{l \leq i \leq m} a_i + \sum_{m+1 \leq i \leq u} a_i \right.$$

Estimar una suma discreta

Reemplazar la suma con una integral

$$1 + 2 + \dots + N. \qquad \sum_{i=1}^N i \sim \int_{x=1}^N x \, dx \sim \frac{1}{2} N^2$$

$$1 + 1/2 + 1/3 + \dots + 1/N. \qquad \sum_{i=1}^N \frac{1}{i} \sim \int_{x=1}^N \frac{1}{x} dx = \ln N$$

$$\sum_{i=1}^N \sum_{j=i}^N \sum_{k=j}^N 1 \sim \int_{x=1}^N \int_{y=x}^N \int_{z=y}^N dz \, dy \, dx \sim \frac{1}{6} N^3$$

Ejercicio 1 para clase

Calcule $T(N)$ y O para cada uno de los fragmentos de código

Ejercicio #1

```
sum = 0;

for ( i=1; i<=n ; i++ )
    for (j=1; j<=n; j++ )
        sum++;
```

Ejercicio #4

```
sum = 0;

while ( n >= 1 ) {
    sum++;
    n=n/2;
}
```

Ejercicio #2

```
sum = 0;

for (k=1; k<=n; k*=2)
    for (j=1; j<= n; j++)
        sum++;
```

Ejercicio #5

```
sum = 0;

while ( n >= 1 ) {
    sum++;
    n=n/3;
}
```

Ejercicio #3

```
sum = 0;

for (i=1; i<=n; i++)
    for (j=1; j<=i; j++)
        sum++;
```

Ejercicio #6

```
sum = 0;

for (i=n; i>=1; i/=2)
    for (j=1; j<=i; j++)
        sum++;
```

Ejercicio 2 para clase

Considere el algoritmo de ordenación por selección.
Encuentre la $T(n)$ y la $O(n)$ en el mejor y peor caso.

```
void <T extends Comparable<T>> T ord_selección ( T [] a ) {  
    int min;  
    n = a.length;  
    for ( int i = 0 ; i < n; i++ ) {  
        min = i;  
        for ( int j=i+1; j < n; j++ )  
            if ( s[j].compareTo(s[min]) < 0 )  
                min = j;  
        swap(a,i,min);  
    }  
}
```

Bibliografía

- Weiss, Mark. A. (2013). Estructura de Datos en Java. 4ta. Edición. Pearson. (Capítulo 5 - *Análisis de Algoritmos*).
- Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). *Introduction to algorithms*. 3ra. Edición. Cambridge, London: MIT Press. (Capítulo 2 - *Getting Started* - 2.2 *Analyzing algorithms* y 3 - *Growth of Functions*).
- Shaffer, C. (2013). Data Structures and Algorithm Analysis. Java version. (Capítulo 3 - *Algorithms Analysis*).
Accesible <http://people.cs.vt.edu/~shaffer/Book/JAVA3elatest.pdf>